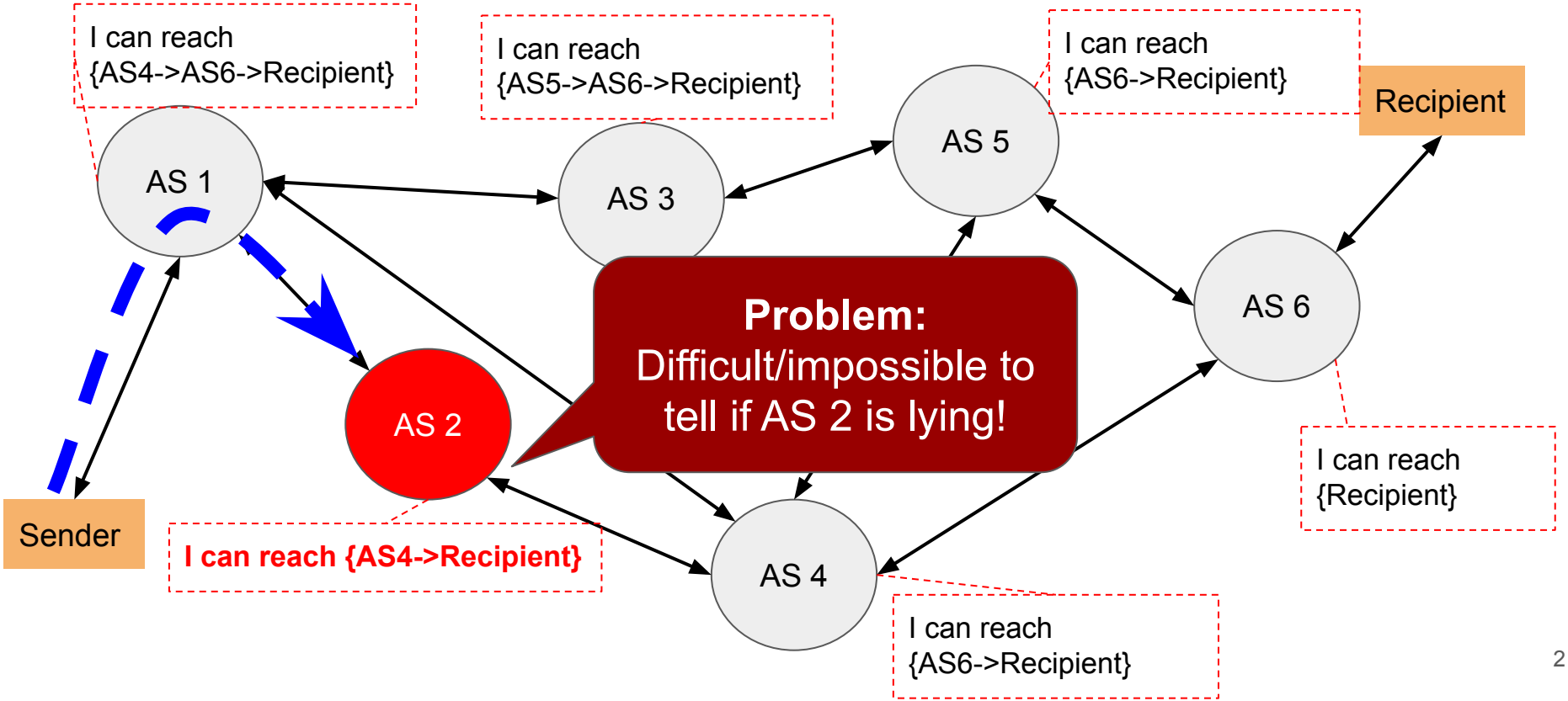
Transport Layer + DNS

CS 161 Spring 2026 - Lecture 27

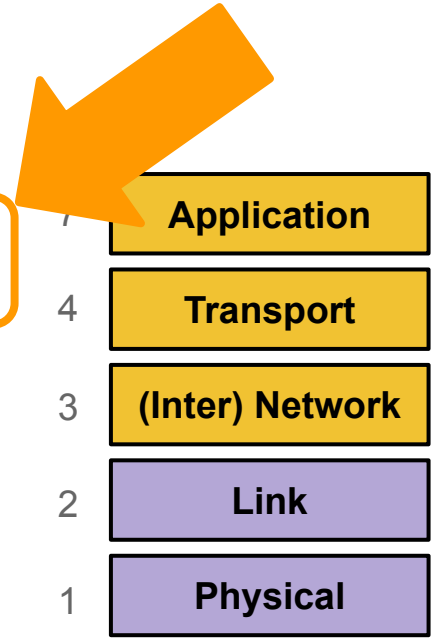
Last time: BGP Hijacking



Reminder: Intro to Networking

We can view network protocols in layers:

- Layer 7: Applications and services — HTTP, SSH, …
  - Address: Depends on protocol — (e.g., for HTTP: /files/stuff.jpg)
- Layer 4: Transport of data — TCP, UDP
  - Address: 16-bit port number — (e.g., port 80)
- Layer 3: Global packet delivery — IP
  - Address: 32-bit string (IPv4), — (e.g., 10.7.12.130)
- Layer 2: Local frame delivery — Ethernet, Wifi
  - Address: 6-byte string (Ethernet) — (e.g., 4e:05:6c:9a:aa:5a)
- Layer 1: Physical communication — Radio waves, light

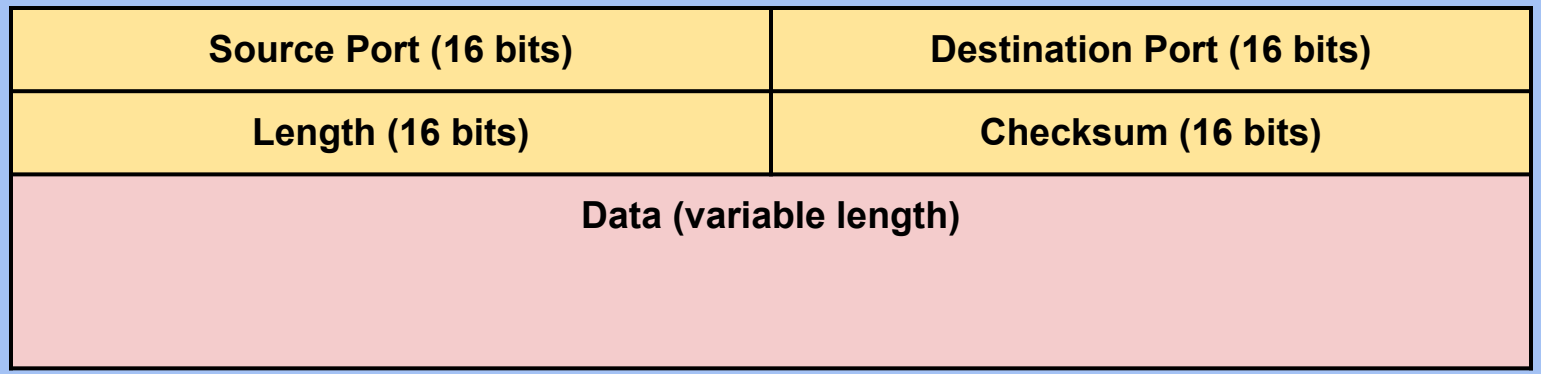


User Datagram Protocol (UDP)



Textbook Chapter 30

UDP Packet Structure



UDP datagram header

User Datagram Protocol (UDP)

- Provides a **datagram** abstraction
  - Applications break their data into datagrams, sent and received as a single unit
- No reliability or ordering guarantees (it's basically IP with port numbers)
  - Like IP, it provides *best-effort* delivery
- Faster than TCP, for many reasons
  - OS can deliver datagrams to the application as soon as they arrive
  - No need for connection setup
  - Usually used by low-latency, high-speed applications (e.g., video streaming, games)

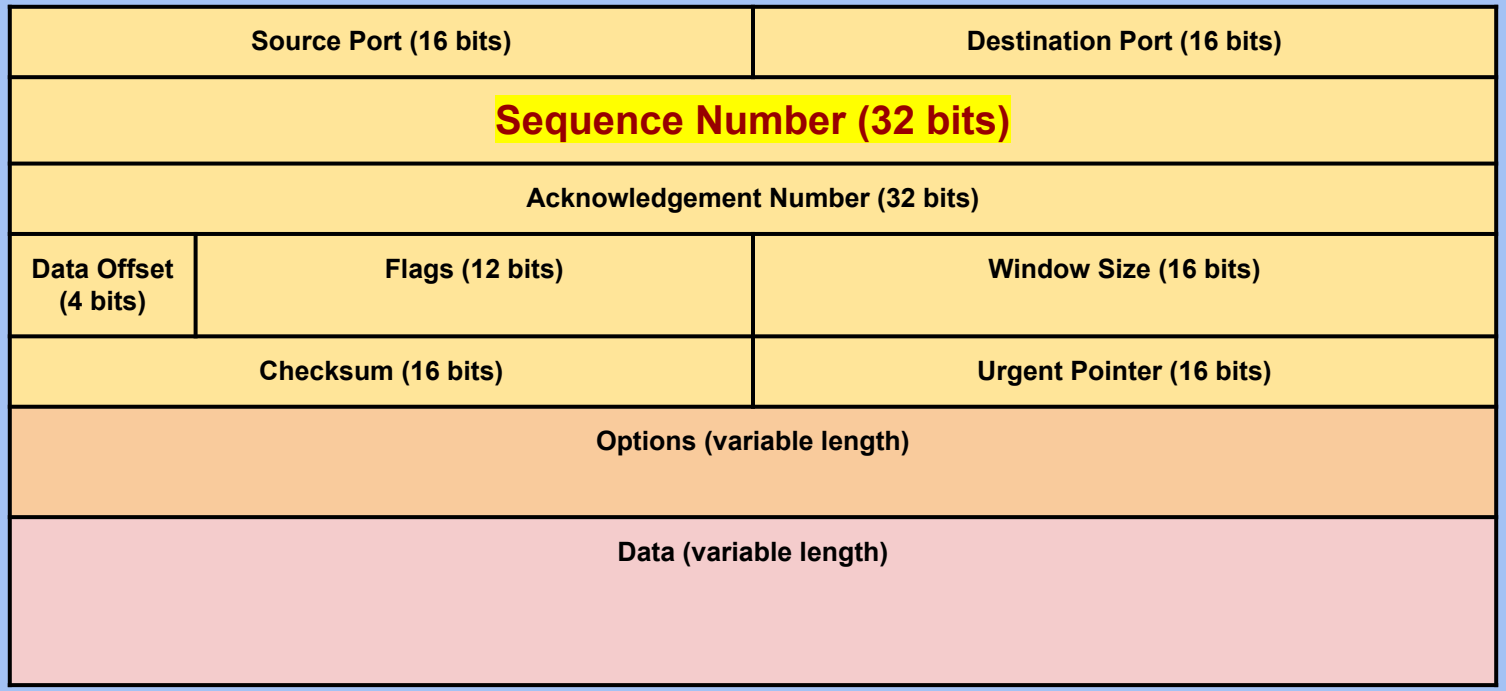
Transmission Control Protocol (TCP)



Transmission Control Protocol (TCP)

- Provides a **byte stream** abstraction
  - If source sends ABCDEFGH the destination gets ABCDEFGH
- To implement, TCP breaks the input stream into **segments**
  - TCP sends each segment as an IP packet
- TCP provides useful guarantees
  - **Ordering:** Segments contain **sequence numbers**, so dest. host can reassemble the stream
  - **Reliability:** Destination sends **acknowledgements** (ACKs) for each segment
    - If the source doesn't receive the ACK, the source sends the packet again
- Many applications use TCP—it's very convenient
  - Writing to a remote host is exactly like writing to a local file on your machine

TCP Packet Structure

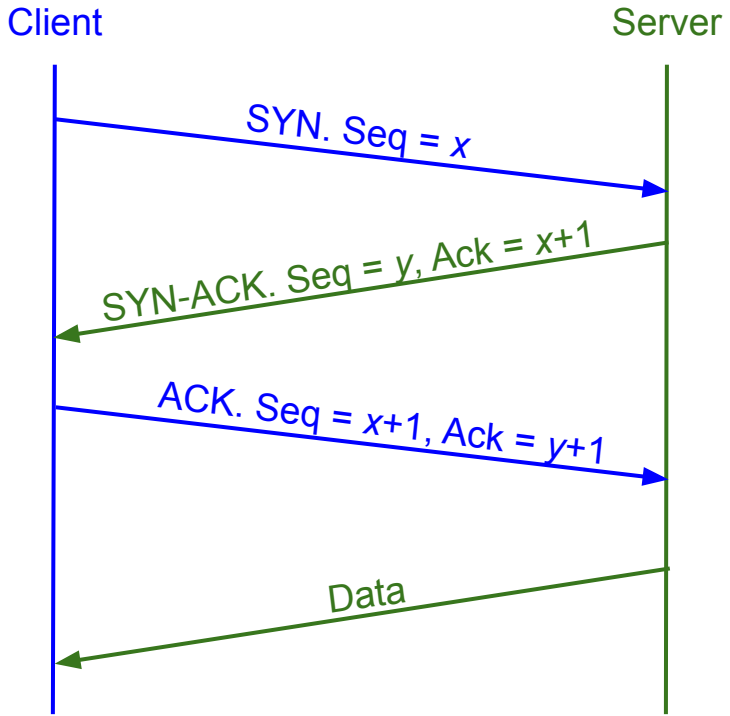


TCP segment header

TCP: 3-Way Handshake

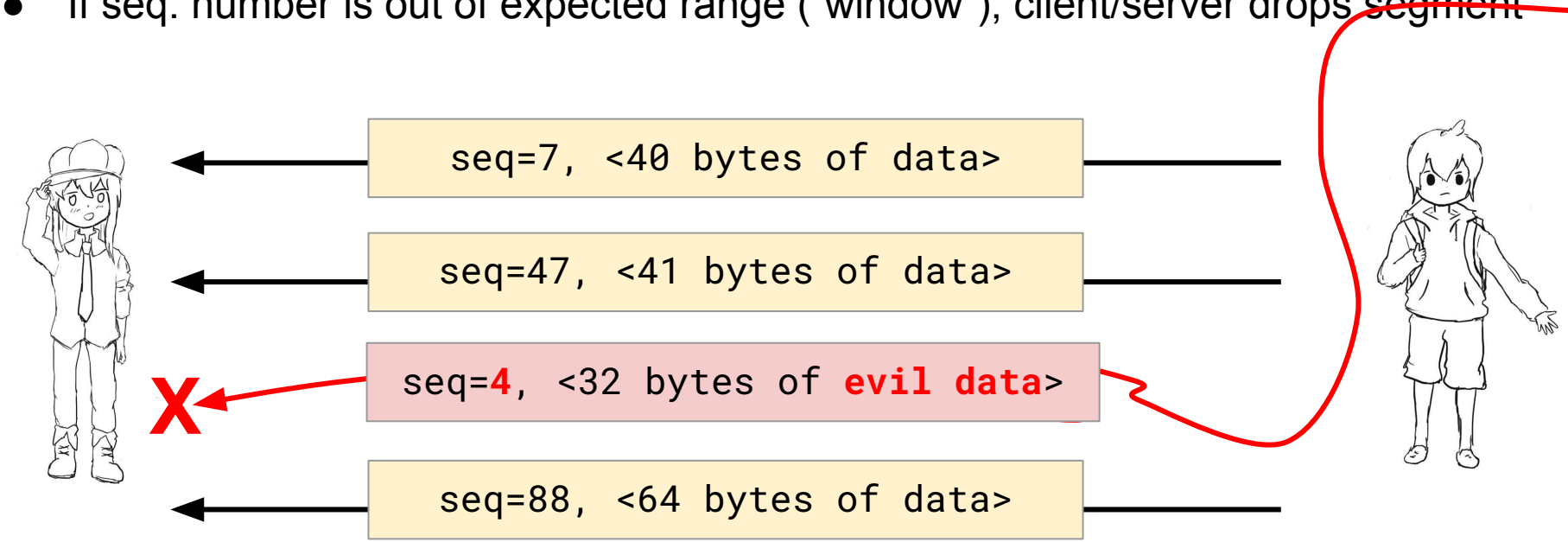
1. Client chooses an initial sequence number *x*
2. Server chooses an initial sequence number *y*
3. Client then returns with an ACK packet
4. The connection is “established.”

⇒ Two round trips to get first byte of data!



Sequence numbers

- During TCP connection setup, client and server pick **random** seq. numbers
- If seq. number is out of expected range (“window”), client/server drops ~~segment~~



Sequence numbers







seq=7, <40 bytes of data>

seq=47, <41 bytes of data>

seq=**88**, <32 bytes of **evil data**>

seq=88, <64 bytes of data>

X

TCP Attacks

- **Data injection**: Spoofing packets to inject malicious data into a connection
  - Need to know: The sender's sequence number
  - **For on-path attackers** → Easy
  - **For off-path attackers** → Must guess 32-bit sequence number +/- small window (hard)

- **RST injection**: Spoofing a RST packet ("connection closed")
  - Forcibly terminates the TCP stream
  - Same requirements as packet injection
  - Easy for on-path and MITM attackers, but hard for off-path attackers
  - Often used in censorship scenarios to block access to sites

“These attacks feel a bit farfetched”

What is QUANTUM?

QUANTUM Generic Animation – High Level of How It Works

1. Target logs into his Yahoo account



Target

Internet Router





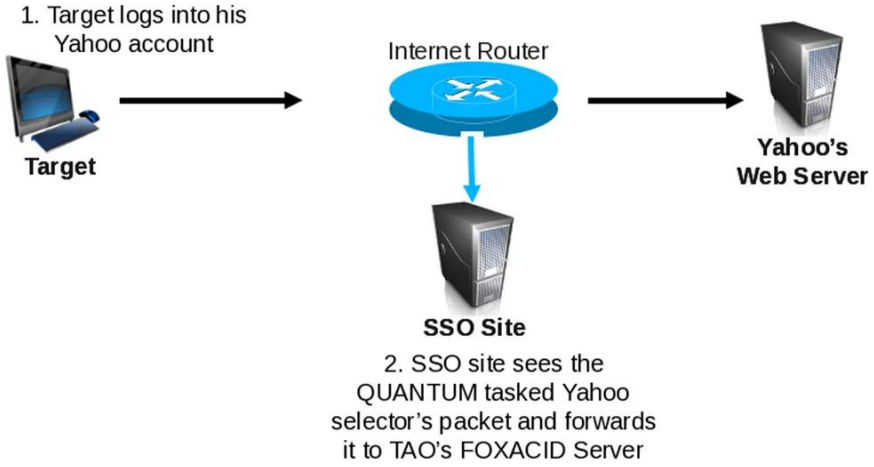
Yahoo's Web Server



SSO Site

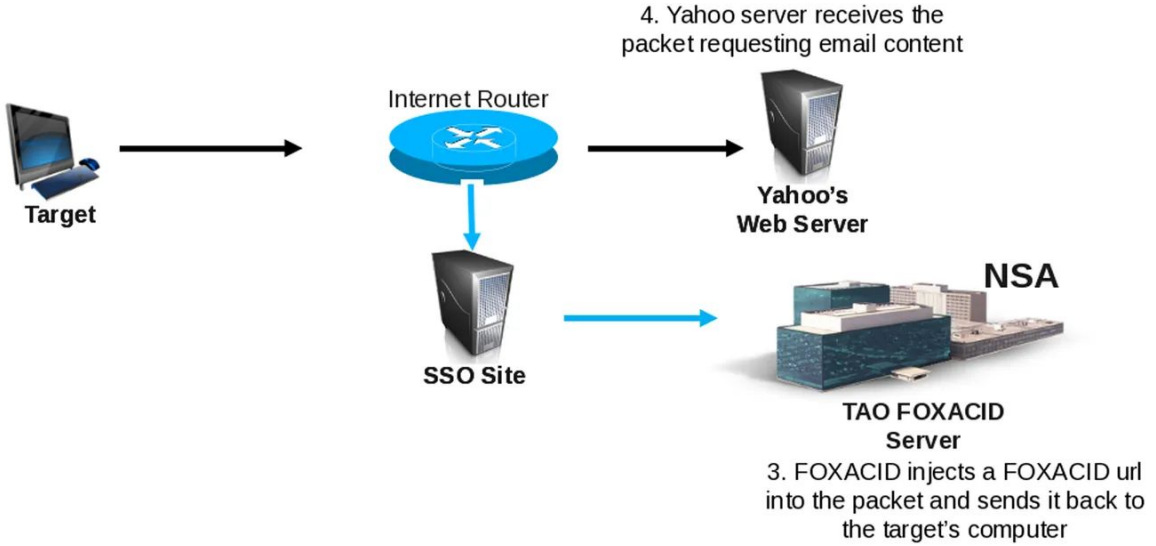
What is QUANTUM?

QUANTUM Generic Animation – High Level of How It Works



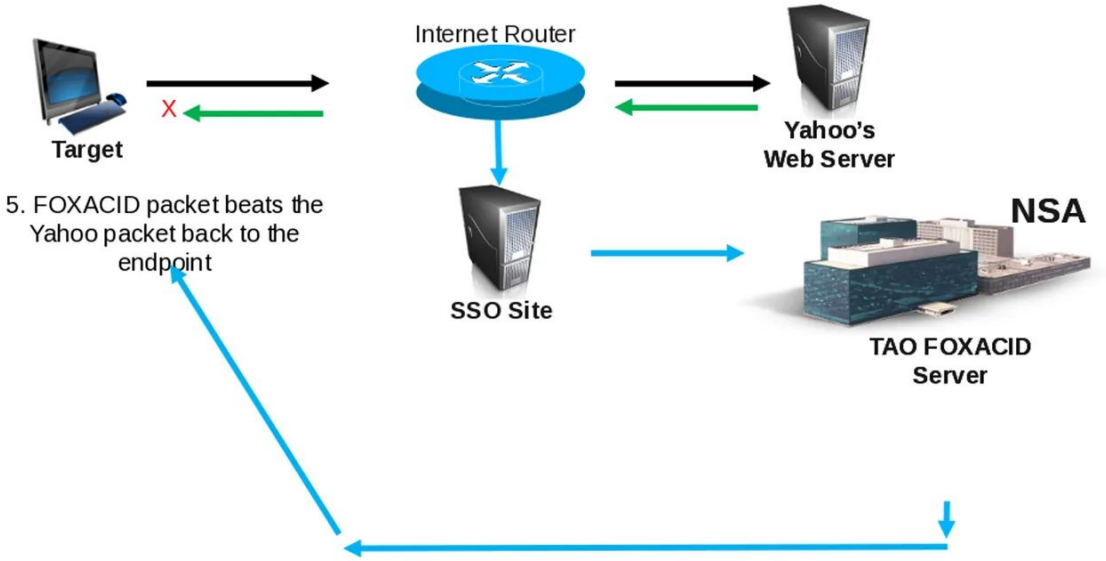
What is QUANTUM?

QUANTUM Generic Animation – High Level of How It Works



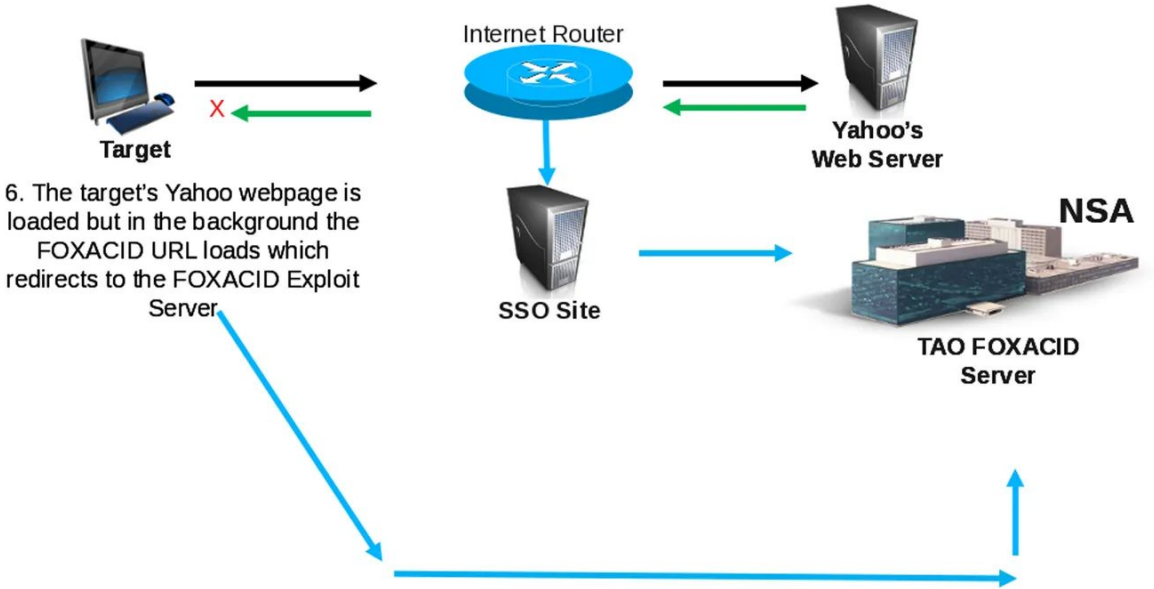
What is QUANTUM?

QUANTUM Generic Animation – High Level of How It Works



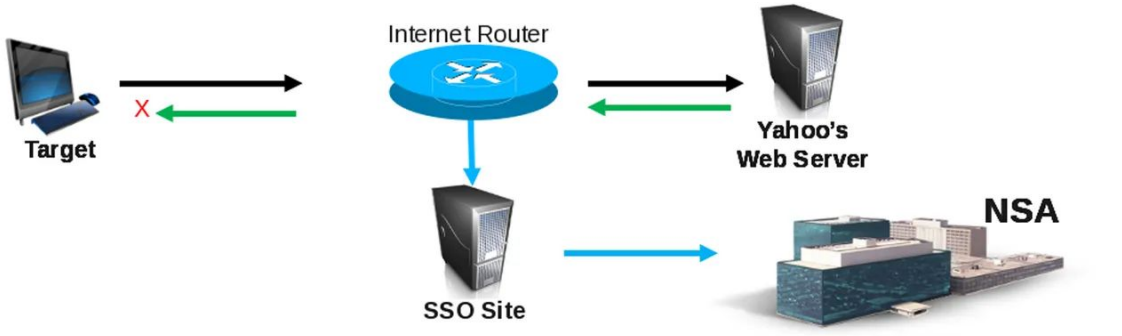
What is QUANTUM?

QUANTUM Generic Animation – High Level of How It Works



What is QUANTUM?

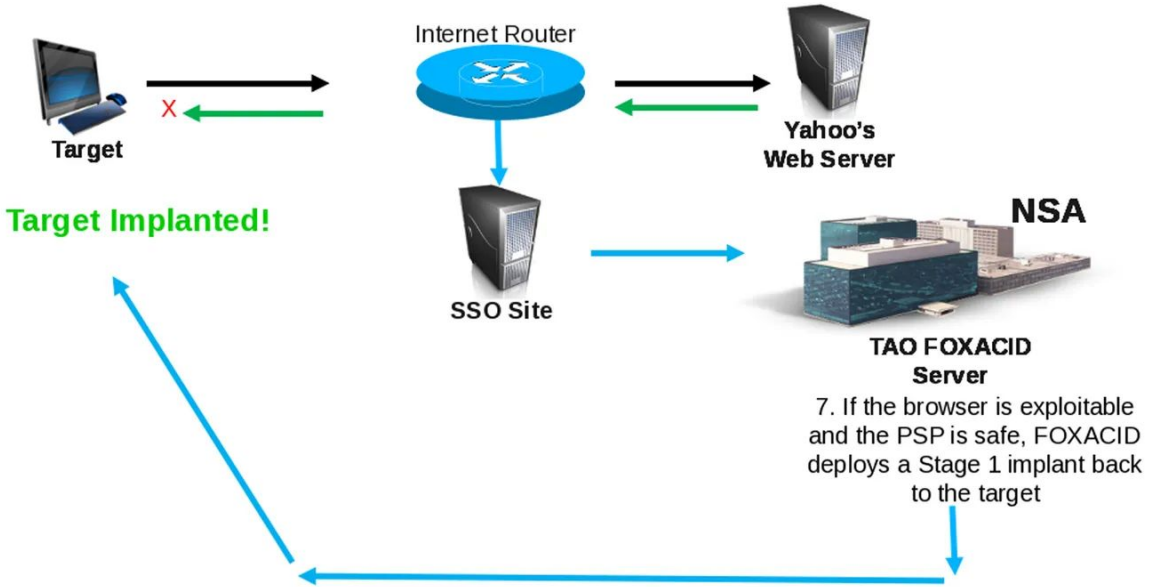
QUANTUM Generic Animation – High Level of How It Works



7. If the browser is exploitable and the PSP is safe, FOXACID deploys a Stage 1 implant back to the target

What is QUANTUM?

QUANTUM Generic Animation – High Level of How It Works



TOP SECRET//SI//REL USA, AUS, CAN, GBR, NZL

QUANTUMTHEORY – GCHQ

If a Partnering Agreement Form (PAF) is set up with GCHQ for the CNO project, then the R&T Analyst can utilize GCHQ QUANTUMTHEORY to include additional capabilities such as:

- ALIBABA
- AOL
- BEBO_EMAIL
- DOUBLE_CLICK
- FACEBOOK_CUSER
- GOOGLE_PREFID
- GMAIL
- HI5
- HOTMAIL
- LINKEDIN
- MAIL_RU
- MICROSOFT_MUID
- MICROSOFT_ANONA
- RAMBLER
- RADIUS
- SIMBAR
- TWITTER
- YAHOO_B
- YAHOO_L/Y
- YANDEX_EMAIL
- YOUTUBE
- IP Address

More information on: https://wiki.gchq/[redacted]/QUANTUM_BISCUIT

If you cannot get to the link try: http://[redacted]

SDS SIGINT | Development | Support

SPIEGEL ONLINE

Summary

- BGP = The protocol routers use to figure out how to route IP traffic
  - **Threat:** BGP Hijacking = Stealing someone else's IP traffic
- UDP = Protocol applications use to send datagram-style traffic over IP
  - Games, video streaming, …
  - **Threat:** Packet injection/tampering
- TCP = Protocol applications use to send bytestream-style traffic over IP
  - Web, SSH, FTP, POP, SMTP, …
  - **Threat:** Packet injection/tampering

BOTTOM LINE

Neither TCP nor UDP provides confidentiality or integrity

Next week: Transport layer security (TLS)

Domain Name System (DNS)



Domain Names

- Computers are addressed by IP address on the Internet
  - Example: `74.125.25.99`
  - Useful for machines: Routers use IP addr to route packets to you (like street address)
  - Not useful for humans: Numbers are not meaningful to humans, hard to remember

- More useful to humans: Human-readable domain names
  - Example: `www.google.com`
  - Not useful for machines: Contains no relevant routing information
  - Useful for humans: Meaningful words and phrases, easy to remember
  - Also: Domain name can stay same even if IP address changes (new ISP, etc.)

Note: Domain names are not URLs. Domain names are part of a URL:

`https://www.google.com/index.html`

DNS: Definition

- **DNS (Domain Name System)**: An Internet protocol for translating human-readable domain names to IP addresses

- Usage
  - You want to send a packet to a certain domain (e.g., you type a domain into your browser)
  - Your computer performs a **DNS lookup** to translate the domain name to an IP address
  - Your computer sends the packet to the corresponding IP address

DNS Resolver

- When you join a Wifi network, your device gets via DHCP:
  - IP address
  - Subnet mask
  - Gateway IP
  - **AND** IP address of a DNS server

- You must have the IP address of DNS server, not domain name

  → Otherwise, you would have a chicken and egg problem

- **Name server**: A server that answers DNS requests
  - Name servers have domain names and IP addresses too
  - Example: Domain `a.edu-servers.net` with IP `192.5.6.30` is a name server

DNS Lookup

- Your computer sends a DNS query: “IP for **`www.google.com`**?”
- DNS server sends an answer: “**`74.125.25.99`**”

- Your computer makes a TON of DNS lookups
  - Every single domain name that you access (web, email, zoom, etc.) requires a DNS query
  - Must be fast!

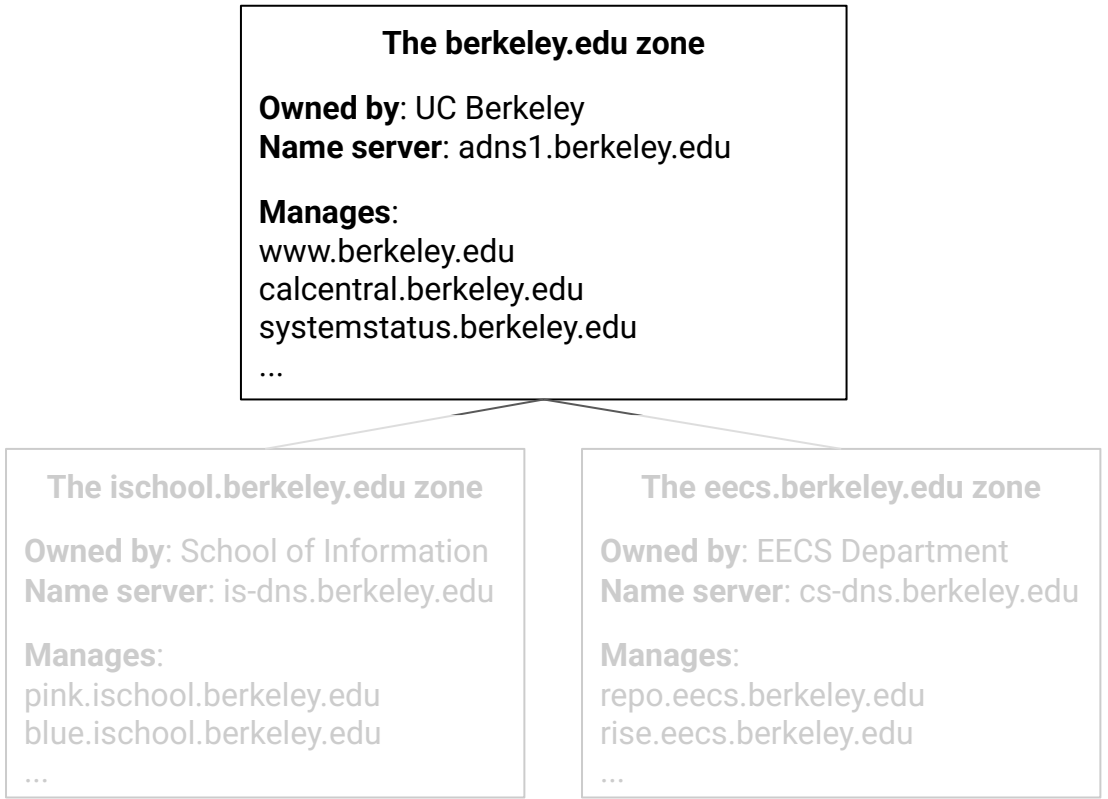
- Performance tricks
  - DNS uses **UDP** – No 3-way handshake!
  - Query and response fit in a single UDP packet
  - **Aggressive caching** once you find out google.com’s IP, remember it for a while (hours)

Different organizations manage different DNS “zones”

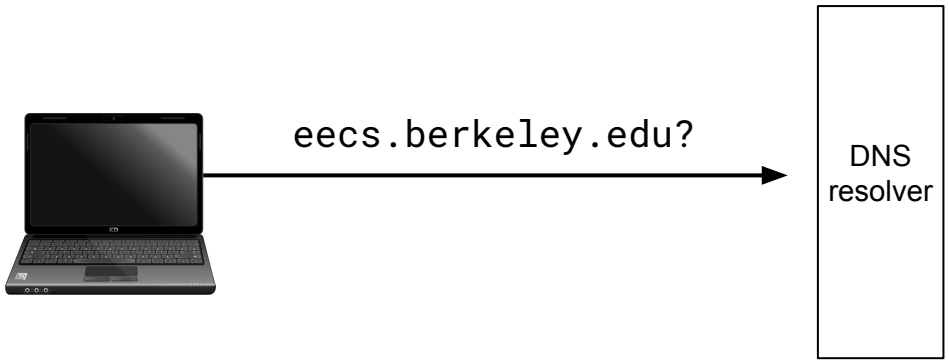
Each node in the tree represents a **zone** of domains.

An organization managing a zone can **delegate** part of its zone to somebody else.

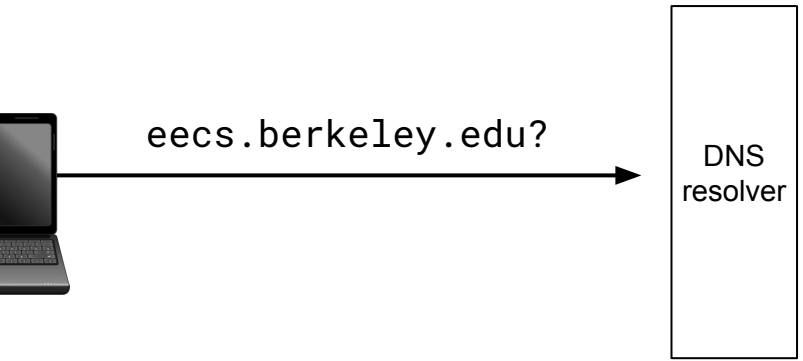
Each zone has an **authoritative name server** that knows about the domains in that zone.



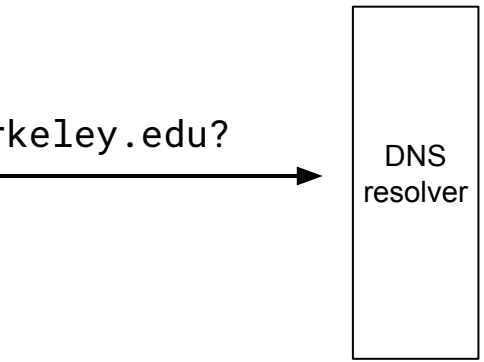
Steps of a DNS Lookup



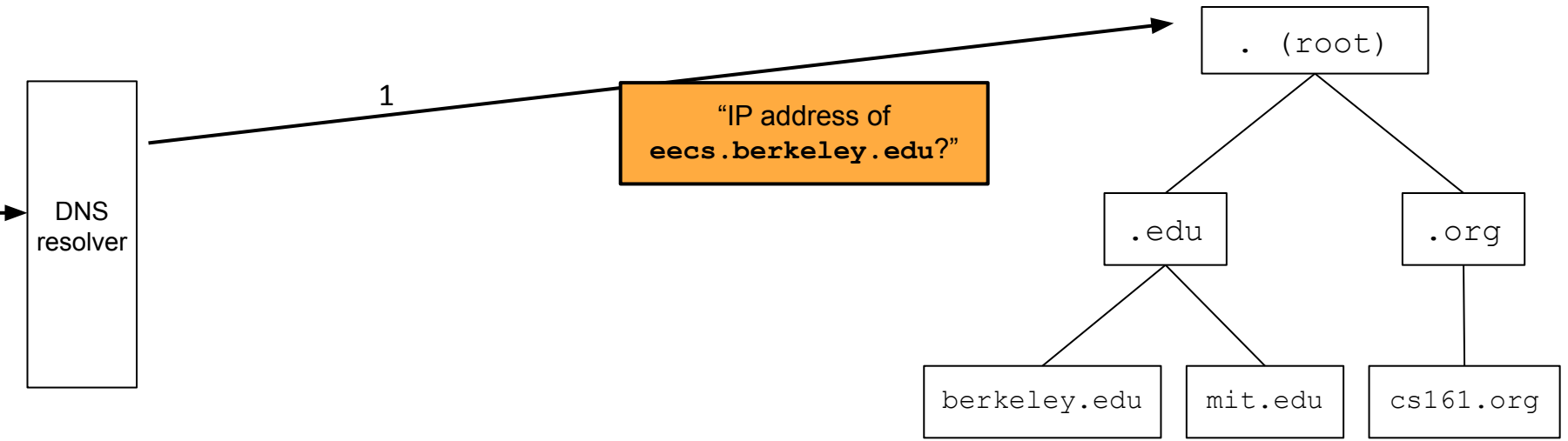
Steps of a DNS Lookup



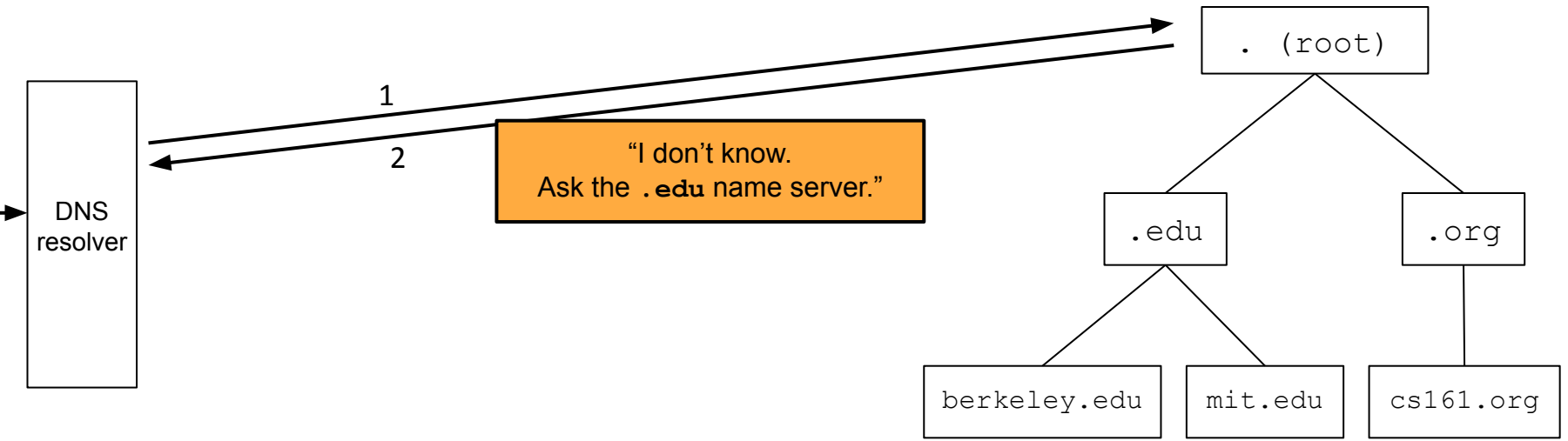
Steps of a DNS Lookup



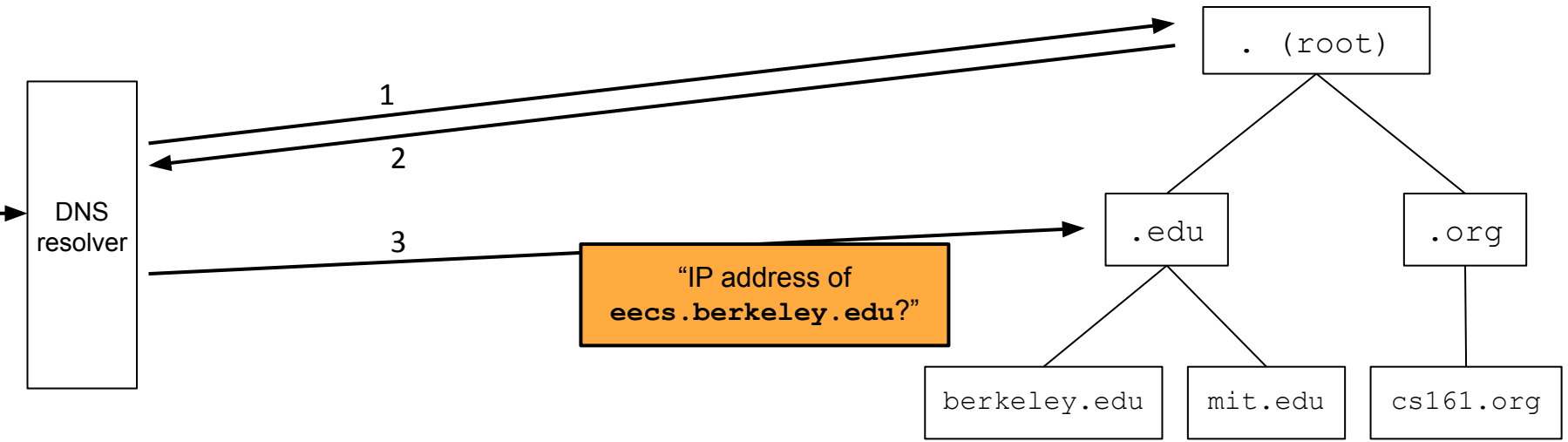
Steps of a DNS Lookup



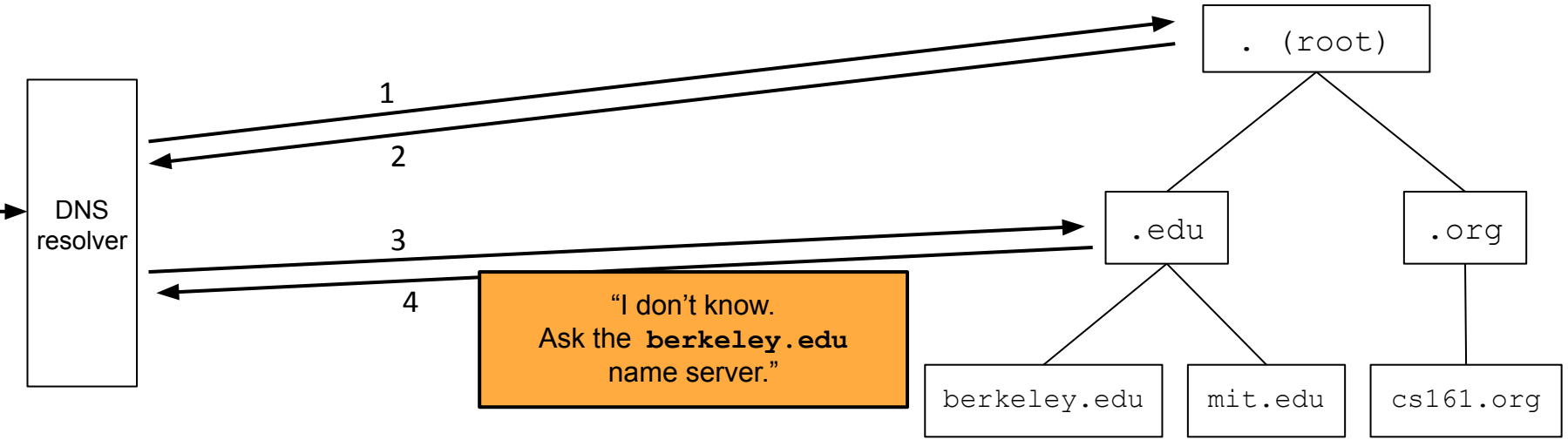
Steps of a DNS Lookup



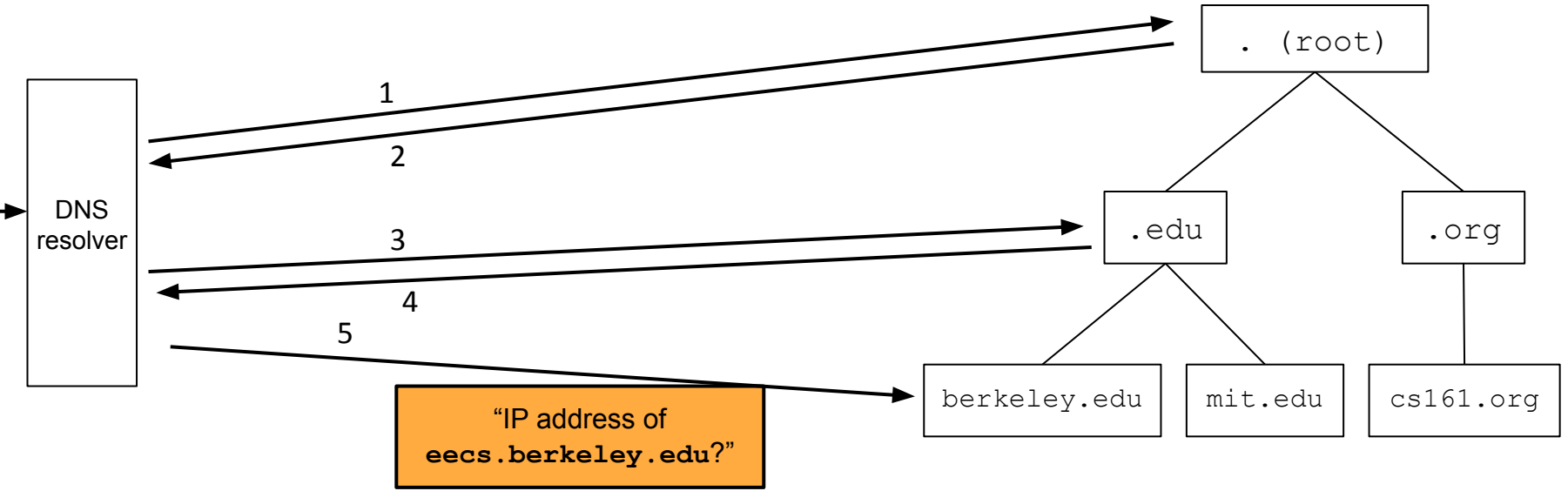
Steps of a DNS Lookup



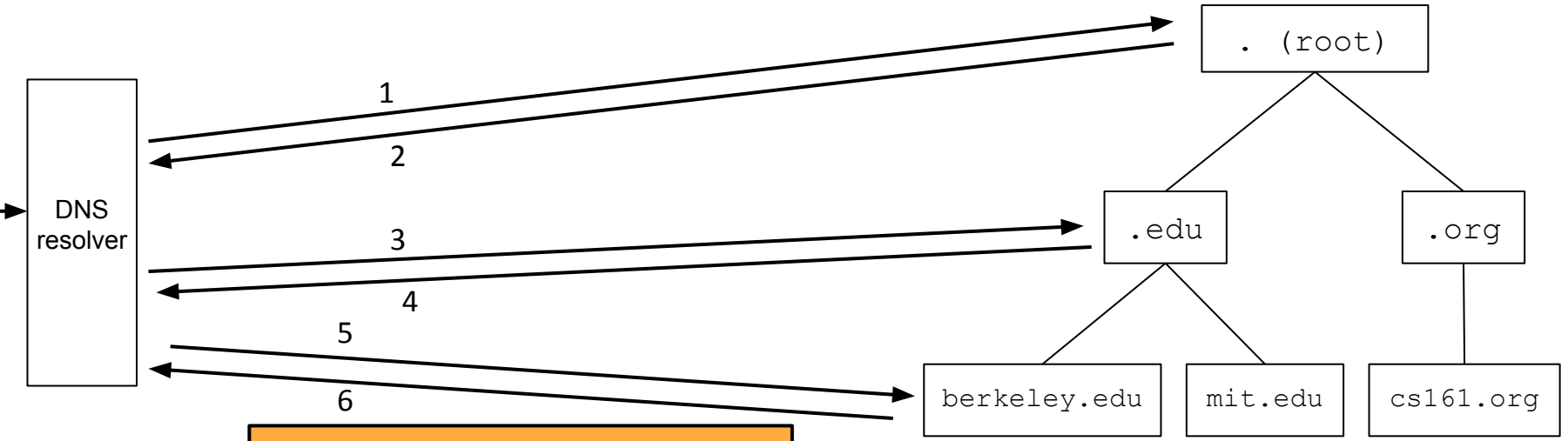
Steps of a DNS Lookup



Steps of a DNS Lookup



Steps of a DNS Lookup



DNS Security

POP QUIZ!

Does UDP provide any authenticity/integrity protection?

In-network attackers can tamper with traffic

- On-path attackers can tamper with DNS traffic
  - Direct honest clients to evil IP addresses
- Clients will **cache** these incorrect responses – attack lasts a while

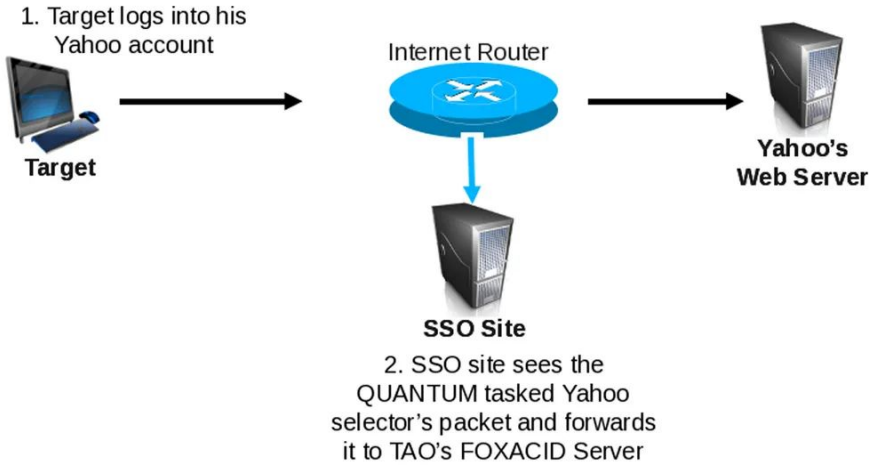
```
;; ANSWER SECTION:
eecs.berkeley.edu.   86400   IN   A   ~~23.185.0.1~~ 6.6.6.6
```



- In the real world, no one messes with DNS traffic, right?

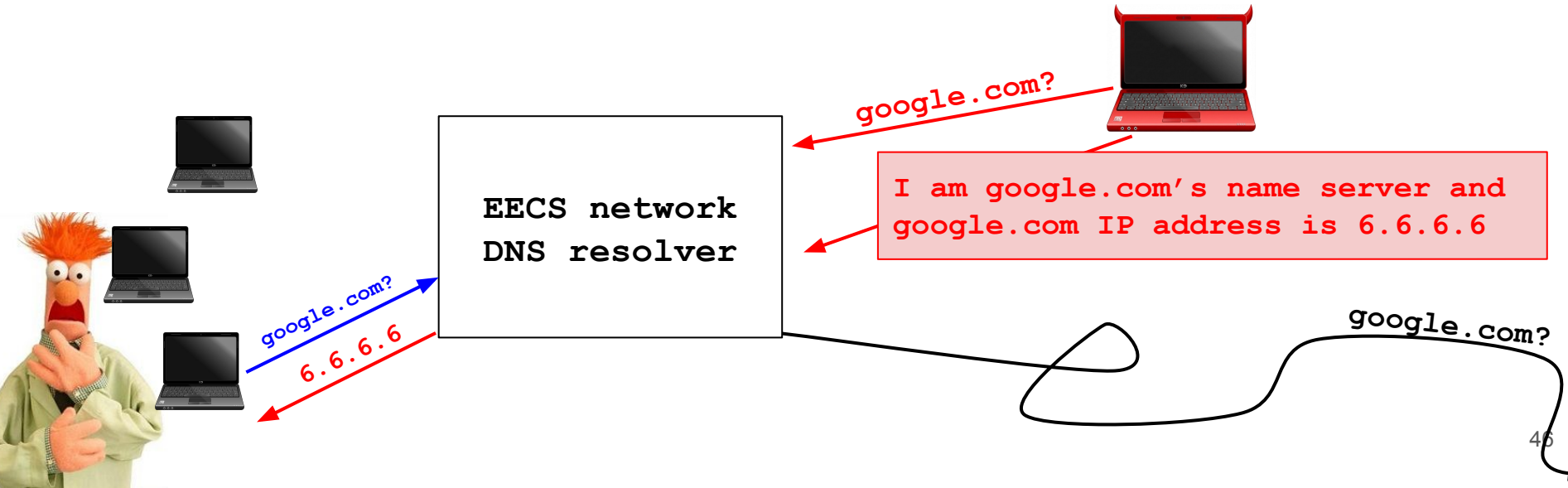
What is QUANTUM?

QUANTUM Generic Animation – High Level of How It Works



DNS Cache-Poisoning Attacks

- **Cache poisoning attack**: Putting evil records in someone's DNS cache
  - Bad if your cache is poisoned
  - Worse if **EVERYONE's** cache is poisoned



Weak defense: ID and source-port randomization

- Each DNS request packet has
  - UDP source port = 16-bit number
  - Transaction ID = 16-bit number

- To defend against cache poisoning: randomize source port and transaction ID
  - Total 2^{32} possibilities = very low probability that off-path attacker will guess correctly
  - Since correct responses are cached, this limits # chances adversary gets to poison

- **On-path attacker can cook DNS responses however it likes**

Stronger defense: DNS over HTTPS (“DoH”)

- Another post-Snowden innovation
- HTTPS provides encryption and authentication for web/HTTP traffic
  - **Idea:** Have client make DNS queries via an HTTPS tunnel
  - On-path attacker can’t mess with DNS responses anymore
- All major browsers use DoH today by default
  - DNS over TLS (“DoT”) is a variant of the same idea

DNS over HTTPS (“DoH”) is imperfect

- Used in your browser but not necessarily your OS
  - So other applications’ DNS queries (mail, Zoom, gaming, chat) may be unprotected
- DoH does not protect the “back end” traffic
  - But Google is better prepared to detect DNS cache poisoning than your local DNS resolver is



Summary: Domain-Name System (DNS)

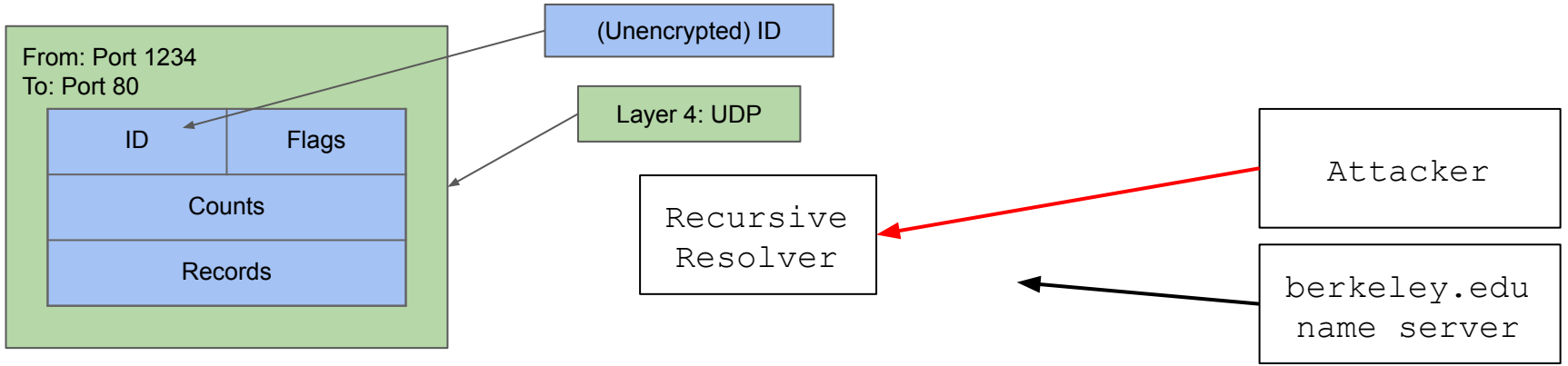
- Translates names into IP addresses
- Name servers are arranged in a domain hierarchy tree
  - Lookups go from root down to the bottom
- DNS traffic goes over UDP → Vulnerable to tampering
- DNS-over-HTTPS (DoH) → More robust to tampering… but not perfect

Next time: The Mirai Botnet

Kaminsky Attack

Security Risk: On-Path Attackers

- DNS is not secure against on-path attackers
- On-path attackers can poison the cache by sending a spoofed response
  - If the spoofed response arrives before the legitimate response, the victim will cache the attacker's malicious records
  - The on-path attacker can see every field in the unencrypted DNS request. Nothing to guess!



Kaminsky Attack

- Now, the attacker can gain more tries at once:
  - The attacker includes
    - `<img src="http://fake1.google.com">`
    - `<img src="http://fake2.google.com">`
    - `<img src="http://fake3.google.com">`
    - `<img src="http://fake4.google.com">`
  - For each, the client makes a request for the domain name
  - The attacker's spoofed response contains:
    - Authority: `fake1.google.com.  172800  IN  NS  www.google.com.`
    - Additional: `www.google.com.  172800  IN  A  6.6.6.6`
  - The client now caches the record for `www.google.com`, and the cache is poisoned!

Kaminsky Attack

- Notice: If the attacker places **`<img src="http://www.google.com">`** multiple times on their website, the browser will only make 1 DNS query
  - The browser caches address of **`www.google.com`**
  - The attacker only gets one try
- Dan Kaminsky, security researcher, noticed that DNS clients would cache additional glue records as if they were authoritative answers, even though they aren't

Kaminsky Attack: Example

- Mallory includes the following image tags on their website
  - `<img src="http://fake1.google.com">`
  - `<img src="http://fake2.google.com">`
  - `<img src="http://fake3.google.com">`
  - `<img src="http://fake4.google.com">`
  - … and so on



- Alice loads Mallory's website
- Alice's browser makes a DNS request for **`fake1.google.com`**
  - We'll show you an example of what happens during this step
- Alice's browser makes a DNS request for **`fake2.google.com`**
- … And so on. Alice's browser continues to make DNS requests for each tag on the website

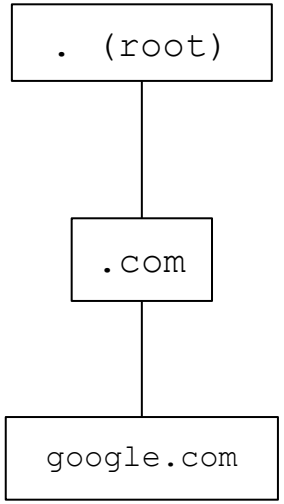


Kaminsky Attack: Example



Stub Resolver

Let's walk through the DNS query for the IP address of **`fake1.google.com`**.



Recursive Resolver

Recursive Resolver Cache	

We'll use this color to represent normal packets across Alice's DNS query.

We'll use this color to represent Mallory's packets across Alice's DNS query.



Kaminsky Attack: Example



Stub Resolver

"Can you figure out what the ip address of **fake1.google.com** is?"

1

Recursive Resolver

Recursive Resolver Cache	

. (root)

.com

google.com



Kaminsky Attack: Example



Stub Resolver

1

Recursive Resolver

Recursive Resolver Cache	

"What is the IP address of **`fake1.google.com`**? This request has ID 161."

2

. (root)

.com

google.com



Kaminsky Attack: Example



Stub Resolver

1

Recursive Resolver

Recursive Resolver Cache	
a.gtld-servers.net.	192.5.6.30

2

3

. (root)

.com

google.com

"I don't know, but I have delegated authority to the .com name server. This response has ID 161.
Authority: .com NS a.gtld-servers.net.
Additional: a.gtld-servers.net. A 192.5.6.30"

The .com name server a.gtld-servers.net. is cached with IP 192.5.6.30.



Kaminsky Attack: Example



Stub Resolver

1

Recursive Resolver

Recursive Resolver Cache	
a.gtld-servers.net.	192.5.6.30

2

3

4

“What is the IP address of **`fake1.google.com`**? This request has ID 972.”

. (root)

.com

google.com



Kaminsky Attack: Example



Stub Resolver

1

Recursive Resolver

Recursive Resolver Cache	
a.gtld-servers.net.	192.5.6.30
ns1.google.com.	**216.239.32.10**

2

3

4

5

. (root)

.com

google.com

"I don't know. But I have delegated authority to the **google.com** name server. This response has ID 972.
Authority: **google.com** NS **ns1.google.com**
Additional: **ns1.google.com.** A **216.329.32.10**"

The **google.com** name server **ns1.google.com.** is cached with IP **216.239.32.10.**



Kaminsky Attack: Example



Stub Resolver

1

Recursive Resolver

Recursive Resolver Cache	
a.gtld-servers.net.	192.5.6.30
ns1.google.com.	216.239.32.10

2

3

4

5

6

. (root)

.com

google.com

"What is the IP address of **fake1.google.com**?
This request has ID 774."



Kaminsky Attack: Example



Stub Resolver

1

Let's see what happens when Mallory correctly guesses the ID number and wins the race condition.

. (root)

2

3

.com

4

5

Recursive Resolver

Recursive Resolver Cache	
a.gtld-servers.net.	192.5.6.30
ns1.google.com.	216.239.32.10

"**fake1.google.com** doesn't exist! This response has ID **774**."

google.com

7

"I don't know. But I have delegated authority to the **fake1.google.com.** NS. This response has ID **774**.
Authority: **fake1.google.com.** NS **www.google.com**
Additional: **www.google.com.** A **6.6.6.6**"

7



Kaminsky Attack: Example



Stub Resolver

1

Recursive Resolver

Recursive Resolver Cache	
a.gtld-servers.net.	192.5.6.30
ns1.google.com.	216.239.32.10
www.google.com.	6.6.6.6

Mallory correctly guessed the ID number and won the race condition, so **www.google.com.** is cached with Mallory's malicious IP **6.6.6.6.**

2

3

4

5

6

. (root)

.com

google.com

"I don't know. But I have delegated authority to the **fake1.google.com.** NS. This response has ID 774.
Authority: **fake1.google.com.** NS **www.google.com**
Additional: **www.google.com.** A **6.6.6.6**"

7



Kaminsky Attack: Example



Stub Resolver

1

We don't really care what next steps are involved in this request, as we have already poisoned the cache.

2

3

. (root)

.com

4

5

6

google.com

Recursive Resolver

Recursive Resolver Cache	
a.gtld-servers.net.	192.5.6.30
ns1.google.com.	216.239.32.10
www.google.com.	6.6.6.6

7



Kaminsky Attack: Example



Stub Resolver

What if Mallory didn't guess the ID number correctly?

. (root)

1

2

3

Recursive Resolver

Recursive Resolver Cache	
a.gtld-servers.net.	192.5.6.30
ns1.google.com.	216.239.32.10

.com

4

5

"**fake1.google.com** doesn't exist! This response has ID **774**."

google.com

7

"I don't know. But I have delegated authority to the **fake1.google.com.** NS. This response has ID **129**.
Authority: **fake1.google.com.** NS **www.google.com**
Additional: **www.google.com.** A **6.6.6.6**"

7



Kaminsky Attack: Example



Stub Resolver

What if Mallory didn't guess the ID number correctly?

. (root)

.com

google.com

1

2

3

4

5

6

7

Recursive Resolver

Recursive Resolver Cache	
a.gtld-servers.net.	192.5.6.30
ns1.google.com.	216.239.32.10

"I don't know. But I have delegated authority to the **fake1.google.com.** NS. This response has ID 129.
Authority: **fake1.google.com.** NS **www.google.com**
Additional: **www.google.com.** A **6.6.6.6**"

7

The recursive resolver discards the response packet from Mallory, because the ID does not match the ID from the request.



Kaminsky Attack: Example



Stub Resolver

1

Recursive Resolver

Recursive Resolver Cache	
a.gtld-servers.net.	192.5.6.30
ns1.google.com.	216.239.32.10
fake1.google.com	**Doesn't exist!**

What if Mallory didn't guess the ID number correctly?

. (root)

.com

google.com

2

3

4

5

6

7

"**fake1.google.com** doesn't exist! This response has ID 774."

What can Mallory do to still poison the cache?



Kaminsky Attack: Example

Mallory can use the DNS requests from her other image tags to try to guess the ID number.

google.com

Recursive Resolver

Recursive Resolver Cache	
a.gtld-servers.net.	192.5.6.30
ns1.google.com.	216.239.32.10
fake1.google.com	Doesn't exist!
…	Doesn't exist!
fake26.google.com	Doesn't exist!

"**fake27.google.com** doesn't exist! This response has ID **161**."

"**fake6.google.com** doesn't exist! This response has ID 427."

"I don't know. But I have delegated authority to the **fake3.google.com.** NS. This response has ID 425.

"I don't know. But I have delegated authority to the **fake4.google.**

"I don't know. But I have delegated authority to the

"I don't know. But I have delegated authority to the **fake27.google.com.** NS. This response has ID 161.
Authority: **fake27.google.com.** NS **www.google.com**
Additional: **www.google.com.** A **6.6.6.6**"



Kaminsky Attack: Example

If Mallory wins the race condition **and** guesses the correct ID during any one of these DNS queries, Mallory's IP will be cached.

Recursive Resolver

Recursive Resolver Cache	
a.gtld-servers.net.	192.5.6.30
ns1.google.com.	216.239.32.10
fake1.google.com	Doesn't exist!
…	Doesn't exist!
fake26.google.com	Doesn't exist!
www.google.com.	6.6.6.6

"I don't know. But I have delegated authority to the **fake27.google.com.** NS. This response has ID 161.
Authority: **fake27.google.com.** NS **www.google.com**
Additional: **www.google.com.** A **6.6.6.6**"

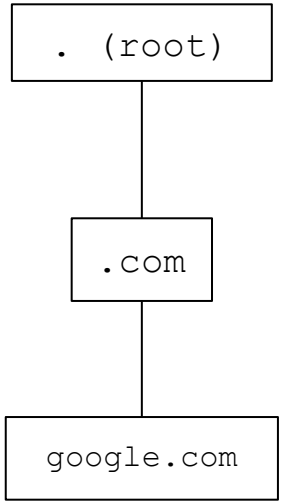


Kaminsky Attack: Example



Stub Resolver

Now that the cache is poisoned, what happens when Alice wants to know the ip address of **www.google.com**?



Recursive Resolver

Recursive Resolver Cache	
a.gtld-servers.net.	192.5.6.30
ns1.google.com.	216.239.32.10
www.google.com.	6.6.6.6



Kaminsky Attack: Example



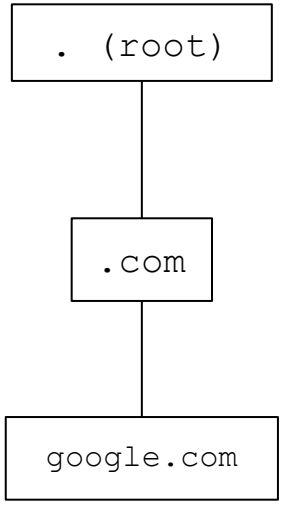
Stub Resolver

"Can you figure out what the ip address of **www.google.com** is?"

1

Recursive Resolver

Recursive Resolver Cache	
a.gtld-servers.net.	192.5.6.30
ns1.google.com.	216.239.32.10
www.google.com.	6.6.6.6





Kaminsky Attack: Example



Stub Resolver

1

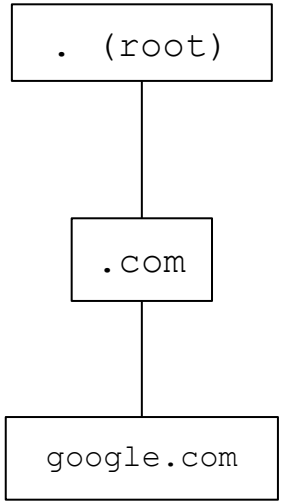
2

"My cache says the ip address of **www.google.com** is **6.6.6.6**."

Recursive Resolver

Recursive Resolver Cache	
a.gtld-servers.net.	192.5.6.30
ns1.google.com.	216.239.32.10
www.google.com.	6.6.6.6

Mallory will receive all traffic Alice meant to send to **www.google.com**!





Pre-Kaminsky Attack: Response Possibilities

	Attacker **correctly** guesses the ID number	Attacker does **not** correctly guess the ID number
Attacker **beats** the race condition against the legitimate NS	The recursive resolver caches a mapping from a legitimate domain name to the attacker's IP address.	Failure. The recursive resolver caches the actual responses and we have to wait until the record expires to try again.
Attacker does **not** beat the race condition against the legitimate NS	Failure. The recursive resolver caches the actual responses and we have to wait until the record expires to try again.	Failure. The recursive resolver caches the actual responses and we have to wait until the record expires to try again.

Kaminsky Attack: Response Possibilities

	Attacker **correctly** guesses the ID number	Attacker does **not** correctly guess the ID number
Attacker **beats** the race condition against the legitimate NS	The recursive resolver caches a mapping from legitimate domain names to the attacker's desired IP addresses.	The recursive resolver ignores the response because the ID does not match the request sent earlier. The recursive resolver caches something saying "This domain does not exist". Try again with another fake domain!
Attacker does **not** beat the race condition against the legitimate NS	The recursive resolver caches something saying "This domain does not exist". Try again with another fake domain!	The recursive resolver caches something saying "This domain does not exist". Try again with another fake domain!